

Date: 12/03/2026

Lab Practical #12:

Implementation of key modular arithmetic operations in C.

1. Write a C program to find GCD of two numbers using the Euclidean algorithm.
2. Write a C program to implement modular exponentiation (for RSA key generation).
3. Write a C program to find the multiplicative inverse of a number modulo n.

Program:

```
// 1. Write a C program to find GCD of two numbers using the Euclidean algorithm.

#include <stdio.h>

int gcd(int a, int b)
{
    while(b != 0)
    {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

int main()
{
    int a, b;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    printf("GCD = %d\n", gcd(a, b));

    return 0;
}
```

Output:

```
Enter two numbers: 48 18
GCD = 6
```



Date: 12/03/2026

Program:

```
// 2. Write a C program to implement modular exponentiation (for RSA key generation).

#include <stdio.h>

long long modExp(long long base, long long exp, long long mod)
{
    long long result = 1;

    while(exp > 0)
    {
        result = (result * base) % mod;
        exp--;
    }

    return result;
}

int main()
{
    long long base, exp, mod;

    printf("Enter base exponent modulus: ");
    scanf("%lld %lld %lld", &base, &exp, &mod);

    printf("Result = %lld\n", modExp(base, exp, mod));

    return 0;
}
```

Output:

```
Enter base exponent modulus: 5 3 13
Result = 8
```

Date: 12/03/2026

Program:

```
// 3. Write a C program to find the multiplicative inverse of a number modulo n.

#include <stdio.h>

int modInverse(int a, int n)
{
    int x;

    for(x = 1; x < n; x++)
    {
        if((a * x) % n == 1)
            return x;
    }

    return -1;
}

int main()
{
    int a, n;

    printf("Enter number and modulo: ");
    scanf("%d %d", &a, &n);

    int inv = modInverse(a, n);

    if(inv == -1)
        printf("Inverse does not exist\n");
    else
        printf("Multiplicative inverse = %d\n", inv);

    return 0;
}
```

Output:

```
Enter number and modulo: 3 11
Multiplicative inverse = 4
```